

■■ API Request Sniffer Engine

Low-Level Design (LLD) & System Architecture Specification | v2.0

System Target: PythonAnywhere WSGI / Docker / Render	Version: 2.0 (IST Timezone)
Author: Engineering Team & Antigravity AI	Storage Mode: Zero-Disk Base64 SQLite Engine
Status: Approved & Production Ready	Date: September 2026

1. Executive Summary & System Goals

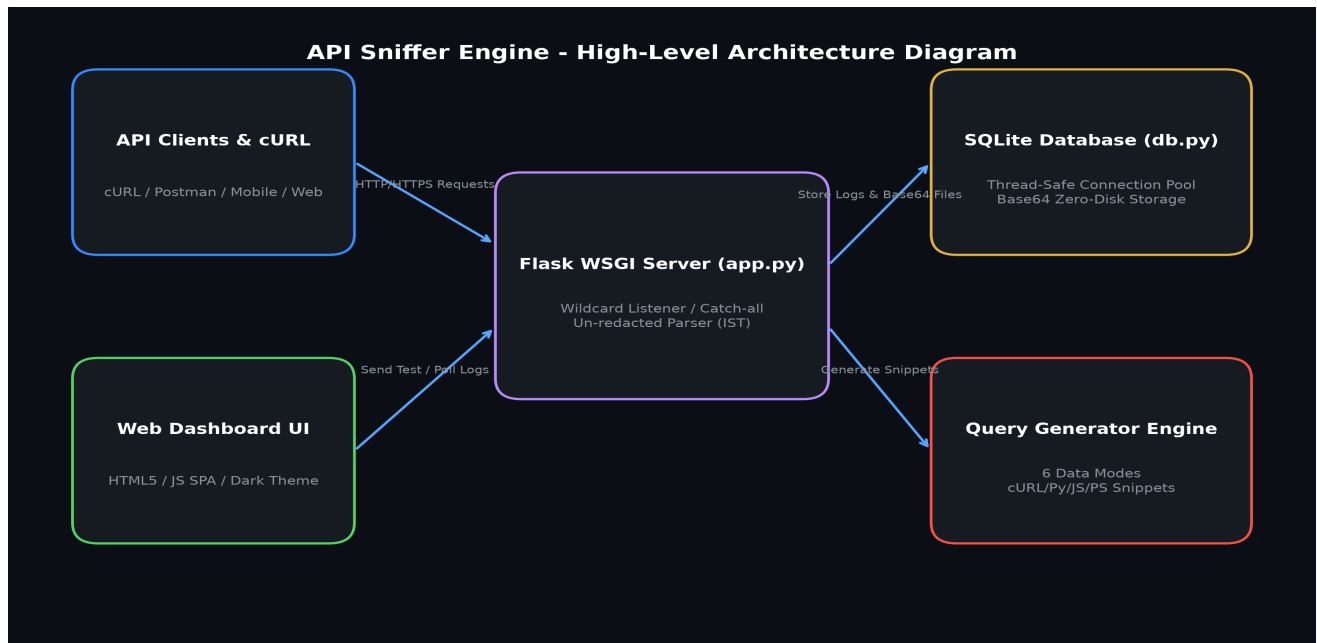
The **API Request Sniffer Engine** is a lightweight, high-performance, self-hosted request inspection and query generator platform. It provides real-time capture, un-redacted logging, and visual inspection of incoming HTTP requests across all standard methods (GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD, TRACE).

Core System Objectives:

- **Un-redacted Logging:** Complete capture of HTTP headers, raw query strings, JSON payloads, form data, client IP, and cookies without masking.
- **IST Timezone Standardization:** Native formatting of all log timestamps in Indian Standard Time (UTC+5:30).
- **Zero-Disk Storage Mode:** Multipart file uploads are Base64 encoded into Data URIs and stored directly in SQLite, eliminating disk quota usage on platforms like PythonAnywhere.
- **Multi-Language Query Generator:** Built-in generator capable of constructing ready-to-run queries across 6 data modes in cURL, Python (requests), JavaScript (fetch), and PowerShell.
- **Minimalist Zero-Lag UI:** Single-page dark dashboard featuring instant search filtering, real-time polling, and interactive in-browser request execution.

2. System High-Level Architecture

The architecture is decoupled into three core layers: an Ingestion/WSGI Server Layer, a SQLite Connection Pool Storage Layer, and a Client Dashboard SPA featuring a multi-language query builder.



3. Low-Level Component Design (LLD)

3.1 Ingestion & Listener Layer (app.py)

The ingestion layer is implemented using Flask wildcard routes (`@flask_app.route('/', methods=['GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'OPTIONS', 'HEAD', 'TRACE'])`). It inspects every incoming request object and extracts raw attributes without truncation.

File Handling Subsystem: Incoming files are read directly into memory from `request.files`, hashed with MD5, and converted into Base64 Data URIs: `data:{content_type};base64,{b64_str}`. This guarantees zero byte output to server disk storage.

3.2 Persistence Layer (db.py)

The database module manages an SQLite backend (`sniffer_logs.db`) using a thread-safe connection pool with write locks. All data models are serialized to JSON strings prior to insertion.

3.3 Multi-Language Query Generator Engine

The client-side generator dynamically builds code snippets for 6 identifier types: JSON Payload, Raw Text Body, File Upload, Query Parameters, Form Data, and Empty Requests. The generator supports four target environments:

- **cURL:** Standard CLI syntax with appropriate `-H` headers and `-d / -F` flags.
- **Python:** Native requests code using `json=`, `data=`, or `files=` parameters.
- **JavaScript:** Modern `fetch()` API with async promise resolution.
- **PowerShell:** `Invoke-RestMethod` cmdlet string formatting.

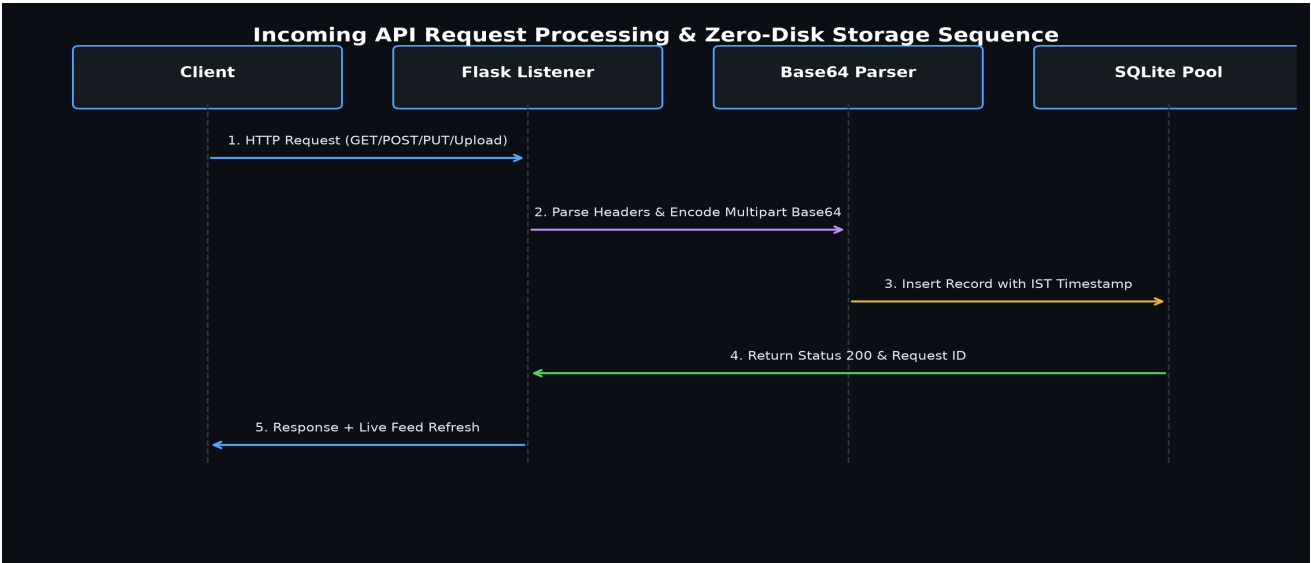
4. Database Schema Specification

Table: sniffer_logs

Field Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique internal log ID
request_id	TEXT	NOT NULL, UNIQUE	UUID v4 string for tracking
timestamp	TEXT	NOT NULL	IST formatted string (YYYY-MM-DD HH:MM:SS)
method	TEXT	NOT NULL	HTTP Method (GET, POST, PUT, etc.)
url	TEXT	NOT NULL	Full incoming URL with query string
path	TEXT	NOT NULL	URL path component
query_params	TEXT (JSON)	DEFAULT '{}'	Parsed query key-value pairs
headers	TEXT (JSON)	NOT NULL	Un-redacted HTTP request headers
body_type	TEXT	NOT NULL	payload / text / file / form / empty
body	TEXT	DEFAULT ''	Raw body or formatted JSON string
files	TEXT (JSON)	DEFAULT '[]'	Base64 data URIs, MD5, metadata
client_ip	TEXT	NOT NULL	Remote IP address of caller

5. Processing Sequence & Data Flow

When an external API call or test request is executed, it traverses the WSGI listener, parses multipart parameters into Base64, persists the payload into SQLite with an IST timestamp, and triggers client-side polling update.



6. Deployment & Environment Strategy

PythonAnywhere WSGI Configuration:

The application runs on PythonAnywhere via WSGI entrypoint forwarding directly to app.py:flask_app without binding background web ports, preserving worker pool safety.

Docker Containerization:

A lightweight multi-stage Docker build is provided for containerized deployments on Render, Railway, or local environments:

```
# Dockerfile snippet
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

7. Verification Matrix & Synthetic Test Suite

The system includes an automated synthetic test suite (test_sniffer.py) verifying all 7 core method variants:

Test Case	Method / Payload	Expected Result	Status
1. Query String	GET /api/v1/test?event=test	200 OK, Parsed Params logged	PASS
2. JSON Body	POST /api/v1/test (application/json)	200 OK, Payload logged	PASS
3. Raw Text	PUT /api/v1/test (text/plain)	200 OK, Raw Text logged	PASS
4. Resource Patch	PATCH /api/v1/test	200 OK, State updated	PASS
5. Delete Request	DELETE /api/v1/test	200 OK, Deletion captured	PASS
6. Preflight	OPTIONS /api/v1/test	200 OK, Headers parsed	PASS
7. File Upload	POST Multipart File Upload	200 OK, Base64 URI saved	PASS